

# Traffic Demand Prediction Solution

This document describes the complete machine learning pipeline built for the Flipkart Gridlock hackathon: preprocessing, feature engineering, model training, validation, and test submission generation.

## 1. Problem Definition

Predict traffic demand (target column: demand, range (0, 1]) for each test row defined by geohash, timestamp, road attributes, and weather on Day 49.

- Evaluation metric: score = max(0, 100 x R2)
- Test set: 41,778 rows; Day 49 timestamps from 2:15 to 13:45 (47 slots)
- Train set: Day 48 full day (69,427 rows) + Day 49 morning only (7,872 rows, 0:00-2:00)
- 89% of test rows have an exact (geohash, timestamp) match in Day 48

## 2. Data Preprocessing

### 2.1 Loading and Cleaning (src/pipeline.py)

- Load train.csv and test.csv from dataset/
- Parse timestamp into minutes since midnight (minutes)
- Decode 6-character geohash to latitude and longitude (pygeohash)
- Extract geohash 5-character prefix (g5) for coarser spatial grouping
- Impute Temperature with train-set median (reused for test)
- Impute NumberofLanes missing values with 0
- Encode categoricals with integer maps; unseen/missing -> Unknown bucket:  
 RoadType (Residential/Street/Highway), LargeVehicles (Allowed/Not Allowed), Landmarks (Yes/No), Weather (Sunny/Rainy/Foggy/Snowy)

### 2.2 Cyclical Time Encoding

- $\text{min\_sin} = \sin(2\pi \cdot \text{minutes} / 1440)$
- $\text{min\_cos} = \cos(2\pi \cdot \text{minutes} / 1440)$

Captures daily periodic demand patterns without treating time as a linear trend.

### 2.3 Leakage-Safe History Construction

- All historical aggregates built from Day 48 only
- Day 48 training rows: leave-one-out (LOO) means to prevent target leakage
- Day 49 and test rows: plain Day 48 means (mirrors inference)

## 3. Feature Engineering

### 3.1 Raw / Structural Features

- lat, lon - decoded geohash coordinates
- minutes, min\_sin, min\_cos - time of day
- NumberofLanes, Temperature
- RoadType, LargeVehicles, Landmarks, Weather (integer encoded)

### 3.2 Historical Prior Features (Day 48 aggregates)

- hist\_g - mean demand per geohash
  - hist\_t - mean demand per timestamp (time-of-day prior)
  - hist\_g5 - mean demand per geohash 5-char prefix
  - hist\_g5t - mean demand per (g5 prefix, timestamp) joint key
  - hist\_nb - spatial neighbor prior (see Section 4)
- hist\_gt - exact Day 48 demand at same (geohash, timestamp). Used only for cross-day inference (Day 49 / test), never as a GBM training feature on Day 48.

### 3.3 Lookup Ladder (baseline predictor)

Fallback chain for rows without a learned model prediction:

- 1. Exact (geohash, timestamp) Day 48 demand
- 2. Optional spatial neighbor at same minute
- 3. Geohash daily mean
- 4. g5 prefix mean
- 5. Timestamp mean
- 6. Global Day 48 mean

Final lookup predictions clipped to [0, 1].

## 4. Spatial Feature Engineering (src/spatial.py)

- Build KDTree index from unique Day 48 geohash coordinates
- For each row: find k=12 nearest geohashes (self excluded)
- neighbor\_predict: mean demand of neighbors at same minute
- If exact minute empty (~58% grid sparsity): widen to +/- 45 minutes
- hist\_nb fills remaining NaNs with hist\_g, then global mean

## 5. Models Implemented

### 5.1 LightGBM Structure Model (src/model.py)

Primary gradient boosting regressor trained on Day 48 features.

- Algorithm: LightGBM regression (RMSE objective)
- Categorical features: RoadType, LargeVehicles, Landmarks, Weather
- reg\_A3 hyperparameters (leaderboard-best baseline):
  - num\_leaves=15, min\_child\_samples=100, lambda\_l2=5.0
  - learning\_rate=0.03, feature\_fraction=0.8, bagging\_fraction=0.8
- Early stopping: 100 rounds on Day 49 morning validation
- Optional log1p target transform (expm1 at predict, clip to [0,1])

### 5.2 Lookup Baselines

- lookup\_plain - ladder without neighbor fallback
- lookup\_nb - ladder with spatial neighbor in fallback chain

### 5.3 Cross-Day Stacked Model (src/crossday\_model.py)

Two-model stack for Day 49 / test inference:

- Matched rows (exact Day 48 key): cross-day GBM with FEATURES + hist\_gt
- Trained on all 7,872 Day 49 morning rows
- Unmatched rows (~11%): main Day 48 GBM (reg\_A3) as fallback
- Final prediction: clip(matched -> crossday, else -> main) to [0, 1]

#### **5.4 Model Tournament (compare\_models.py)**

Nine variants compared on train-only splits:

- lookup\_plain, lookup\_nb
- gbm\_baseline, gbm\_cap511, gbm\_cap1023, gbm\_cap1023\_nolog
- blend\_cap1023, crossday\_stack, hist\_gt\_hybrid

## 6. Validation Strategy

### 6.1 Primary Leaderboard Proxy (validate.py, validate\_crossday\_v2.py)

- Train on Day 48, evaluate on Day 49 morning labels
- 80/20 holdout on Day 49 for cross-day model selection
- Metric: R2 (coefficient of determination)

### 6.2 Additional Holdouts (compare\_models.py)

- random\_holdout - 80/20 Day 48 split, aggregates rebuilt from kept rows
- testhours\_holdout - hold out 2:15-13:45 on Day 48
- day49\_morning - full cross-day evaluation

### 6.3 Honesty Check (validate\_midday.py)

- Within Day 48: compare model skill on morning vs test-hour band
- Conservative estimate of daytime generalization

## 7. Model Selection Results

### 7.1 Cross-Day V2 Tournament (validate\_crossday\_v2.py)

Winner: crossday\_nolog\_all

- Holdout R2: 0.9231
- Full Day 49 R2: 0.9638
- crossday\_iter: 2994, main\_iter: 226
- Both models use identity link (use\_log=False)
- reg\_A3 params for both cross-day and main fallback models

Comparison on Day 49 morning holdout:

- crossday\_nolog\_all: 0.9231 (selected)
- crossday\_log: 0.9211
- crossday\_nolog: 0.9114
- gbm\_baseline: 0.7834

### 7.2 Key Findings

- High-capacity LightGBM (num\_leaves 1023+) memorized Day 48 but scored worse on leaderboard (~86)
- reg\_A3 regularization generalized best to Day 49 (~88.95 leaderboard)
- Removing log1p transform improved cross-day exact recovery
- hist\_g5t joint encoding added finer spatio-temporal prior
- Raw hist\_gt copy alone gives R2 ~0.52; cross-day mapping model required

## 8. Final Test Pipeline (submit\_v2.py)

- 1. Load and preprocess train + test via prepare()
- 2. Build spatial index and all historical features
- 3. Train cross-day model on Day 49 with hist\_gt (crossday\_nolog\_all settings)

- 4. Train main GBM on Day 48 with early-stop on Day 49
- 5. Route predictions: matched rows -> crossday, unmatched -> main
- 6. Write submissions/sub\_best.csv and sub\_crossday\_v2.csv
- 7. Also regenerate sub\_gbm.csv (reg\_A3 baseline for A/B)

## 9. Submission Format

- File: CSV with exactly 41,778 rows x 2 columns
- Columns: Index, demand
- Index order must match dataset/test.csv exactly
- demand values in [0, 1], no NaN or Inf
- Primary submission: submissions/sub\_best.csv

## 10. Project File Structure

- src/pipeline.py - loading, cleaning, aggregates, hist features, lookup
- src/spatial.py - KDTree neighbor index and hist\_nb
- src/model.py - LightGBM train/predict, reg\_A3 params
- src/crossday\_model.py - cross-day stack training and inference
- validate.py - Day 48 -> Day 49 morning validation
- validate\_crossday\_v2.py - cross-day variant selection
- compare\_models.py - 9-model tournament on train splits
- submit\_v2.py - final test submission generator
- submissions/ - output prediction CSVs

## 11. Tech Stack

- Python 3.14, pandas, numpy, scikit-learn, lightgbm, pygeohash
- Platform: Windows, PowerShell